

Guide développeur — Wash Manager

Tout ce qu'il faut pour ****naviguer dans le code, continuer le développement et rendre le système pleinement fonctionnel****. À lire en premier.

Documents liés : ARCHITECTURE.md (schémas), DEPLOIEMENT.md (production),
GUIDE_UTILISATEUR.md (fonctionnalités côté utilisateur), HANDOFF_DEV.md
(liste des TODO(dev) restants), schema.sql (schéma SQL de référence).

1. Vue d'ensemble

Système de gestion + monitoring IA de stations de lavage (multi-sites). Trois

briques découplées :

```
ai/           Pipeline caméra (edge) → émet des ÉVÉNEMENTS au backend
backend/     API FastAPI + PostgreSQL → logique métier, décisions, temps réel
frontend/    Dashboard React + Tailwind (PWA)
```

Principe clé : ai/ ne fait que ***percevoir*** (détecter, suivre, lire). Toute la ***décision*** (classification, anomalies, consommation, alertes) est dans backend/. Le frontend/ ne fait qu'***afficher***. Ce découplage permet de développer/tester chaque couche indépendamment.

2. Stack technique

- **Backend** : Python 3.12, FastAPI, SQLAlchemy 2, PostgreSQL, Alembic (migrations), APScheduler (tâches planifiées), ReportLab (PDF), Pillow (images), python-jose (JWT), passlib (pbkdf2_sha256).
- **Frontend** : React 18 + Vite, Tailwind CSS, Recharts, React Router.
- **IA** : ultralytics (YOLO + ByteTrack), OpenCV, EasyOCR/PaddleOCR (à brancher).

3. Démarrer en local

```
cp .env.example .env
# Tout en Docker :
docker compose up -d --build
docker compose exec backend python -m app.db.seed          # données de référence
docker compose exec backend python -m app.db.demo 7 25    # (option) données de démo

# OU à la main :
cd backend && python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
alembic upgrade head && python -m app.db.seed
uvicorn app.main:app --reload          # http://localhost:8000/docs
cd ../frontend && npm install && npm run dev  # http://localhost:5173
```

Compte par défaut : admin@wash.local / changeme.

Développer sans caméras : python -m app.db.demo (données) ou

cd ai && python simulator.py --count 30 (émet de vrais événements).

4. Arborescence commentée

```
backend/app/
|-- main.py           Point d'entrée FastAPI : monte les routes, /media, scheduler, WebSocket
|-- scheduler.py      Jobs planifiés : rapport journalier, scan des absences
|-- core/
|   |-- config.py     Settings (.env)
|   |-- database.py   Engine + session + Base ORM
|   |-- security.py   JWT, hachage, vérif clé d'ingestion IA
|-- models/           Tables SQLAlchemy (une par fichier) – voir §5
|-- schemas/          Schémas Pydantic (I/O API) ; event.py = contrat IA↔backend
|-- api/
|   |-- deps.py       Dépendances : get_current_user, require_role, resolve_site (multi-site)
|   |-- routes/       Un fichier par ressource – voir §6
|-- services/         LOGIQUE MÉTIER – voir §7
|-- db/
|   |-- seed.py       Données de référence (forfaits, sites, admin...)
|   |-- demo.py       Jeu de démo multi-jours

backend/alembic/      Migrations (env.py branché sur les modèles + .env)

ai/
|-- run.py            Lanceur du pipeline edge (multi-caméras)
|-- simulator.py      « Fausse caméra » : émet des événements réalistes
|-- detect_video.py   Détection+suivi+comptage sur une VIDÉO (test sans caméras)
|-- config.yaml       Caméras, zones, connexion backend
|-- pipeline/         detector, tracker, lpr, zones, vest, camera, orchestrator, events

frontend/src/
|-- api/client.js     Toutes les URLs backend (point de raccordement unique)
|-- context/          AuthContext (rôle/site), LiveContext (WebSocket temps réel)
|-- components/       Sidebar, cartes, graphiques, éditeurs réutilisables
|-- pages/            Un écran par fichier – voir §8
```

5. Modèle de données (models/)

vehicules, employes, forfaits, sites, bays, transactions, tickets,

anomalies, pointages, produits, forfait_produits (recette de consommation),

horaires_site, horaires_employe, parametres, rapports_journaliers,

utilisateurs. Détail des colonnes : docs/schema.sql. Enums : models/enums.py.

L'entité centrale est transactions : un passage de véhicule (temps par zone,

forfait détecté vs payé, conformité, baie/site, anomalies).

6. Les routes API (api/routes/)

Fichier	Ce qu'il expose
auth.py	login (JWT), /auth/me (profil)
users.py	gestion des comptes (admin only)
events.py	ingestion des événements IA (clé X-AI-Key)
tickets.py	caisse : créer/annuler/rapprocher un ticket
queue.py	file d'attente + mode manuel (démarrer/terminer)
sites.py, bays.py	sites et baies (état, staff)
dashboard.py	KPI, aperçu, wash-details, graphiques, en-cours
transactions.py, vehicules.py	historique, véhicules
employes.py	employés, performance, présence

Fichier	Ce qu'il expose
pointage.py	pointage selfie horodaté
payments.py	vue financière Paid/Pending
produits.py	inventaire (stock, mouvements)
forfaits.py	forfaits + recette de consommation
horaires.py	horaires ouverture site / travail employé
anomalies.py	anomalies (avec emplacement)
rapports.py, parametres.py	rapports PDF, paramètres réglables
live.py	WebSocket /ws/live (temps réel)

Toutes les routes protégées dépendent de `get_current_user`. Le filtrage

multi-site se fait via `resolve_site` (`deps.py`) : un manager/caissier est

forcé sur son site quel que soit le `site_id` demandé.

7. Où vit la logique métier (services/) — le plus important

Service	Rôle
ingestion.py	Machine à états : applique les événements IA → construit les transactions (<code>finaliser_transaction</code> = classification + rapprochement + anomalies + consommation stock). Point d'entrée : <code>traiter_evenement</code> .
classification.py	Déduit le forfait effectué depuis les zones/durée (pilote par la base).
anomalie.py	Règles d'anomalies (forfait non respecté, non facturé, ticket fantôme...).
reconciliation.py	Rapproche un ticket de caisse d'une transaction (plaque puis temps).
presence.py	Présence / ponctualité / productivité (planning + pointages + lavages).
alertes.py	Alertes RH retard/absence (anomalie + temps réel).
rapport.py	Agrégats du jour + génération PDF.
photo.py	Selfies de pointage horodatés (filigrane).
notification.py	Envoi WhatsApp (à brancher avec vos identifiants).
parametres.py	Lecture/écriture des seuils configurables.

8. Le frontend (pages/)

Login, Dashboard, Caisse, Queue, Bays, Employees, Pointage, Presence, Payments,

Inventory, Anomalies, History, Reports, Config, Users, Vehicules, LiveView,

Rapprochement. **Toutes les URLs backend sont dans** `src/api/client.js` — pour

câbler une page, ajoutez-y une méthode. Le temps réel : `LiveContext` incrémente

un compteur version à chaque message WebSocket ; les composants l'ajoutent à leurs deps d'effet pour se rafraîchir.

9. Flux de données (à comprendre absolument)

```
Caméra/Simulateur --event--■ POST /events --■ ingestion.traiter_evenement
(ENTREE, ZONE_ENTER/EXIT, PLAQUE, BADGE, SORTIE)
|
à la SORTIE : finaliser_transaction()
|- classification du forfait effectué
|- rapprochement du ticket de caisse
|- détection + persistance des anomalies
|- consommation d'inventaire (recette forfait)
`- diffusion WebSocket (le dashboard se rafraîchit)
```

En **mode manuel** (sans caméras), c'est queue.py (démarrer/terminer) qui crée et clôture les transactions au lieu des événements.

10. Ajouter une fonctionnalité (patterns)

- **Nouveau champ/table** : ajouter/modifier un modèle dans models/, l'importer dans models/__init__.py, puis alembic revision --autogenerate -m "... " et alembic upgrade head. Mettre à jour schema.sql (doc).
- **Nouvel endpoint** : créer/éditer un fichier dans api/routes/, le monter dans main.py. Pour un filtrage par site, ajouter site_id=Depends(resolve_site).
- **Nouvelle page** : créer pages/X.jsx, ajouter la route dans App.jsx, une entrée dans Sidebar.jsx, et les méthodes dans api/client.js.
- **Piège React** : ne jamais passer à useEffect une fonction qui **retourne une Promise** (useEffect(() => { charger(); }, []), pas useEffect(charger, [])).

11. Tests

```
cd backend && pytest tests      # logique métier (classification, anomalies, rapprochement)
cd ai && pytest tests           # géométrie des zones, matching couleur de gilet
cd frontend && npm run build    # vérifie que tout compile
```

La logique déterministe (services) est testée sans base de données. **À compléter** (TODO(dev)) : tests d'intégration des routes API.

12. Le pipeline IA — état et ce qui reste

- **Fonctionnel** : detector.py (YOLO), tracker.py (ByteTrack), camera.py (RTSP ou fichier vidéo), zones.py (géométrie), vest.py (couleur de gilet), detect_video.py (test sur vidéo), simulator.py. Le contrat d'événements (ai/pipeline/events.py ↔ backend/app/schemas/event.py) est stable.
- **À finir (nécessite les caméras)** : fournir les poids YOLO (ai/models/), brancher l'OCR des plaques marocaines (lpr.py), calibrer les lignes/zones dans

ai/config.yaml, boucles de frames multi-caméras dans run.py.

13. Ce qui reste à faire (résumé)

Voir docs/HANDOFF_DEV.md pour la liste complète et à jour. Chercher aussi dans

le code : `grep -rn "TODO(dev)" ..` Points principaux : OCR plaques + poids YOLO (matériel), envoi WhatsApp réel, tests d'intégration API, filtrage multi-site sur les écritures.

14. Conventions

- Code et commentaires en français (cohérence avec l'existant).
- Logique métier dans services/, jamais dans les routes ni le frontend.
- Un modèle par fichier ; enums centralisés dans models/enums.py.
- Après toute modif de modèle : générer une migration Alembic (ne jamais `create_all` en production).

15. Modules ajoutés (contrôle & pilotage)

Nouveaux domaines depuis la v1. Chaque fonction suit le même patron :

models/ → services/ → api/routes/ → migration Alembic → tests → page React.

Domaine	Modèle	Service	Route(s)	Page
Marge par forfait	Produit.prix_unitaire, ForfaitProduit	marge.py	/marges	Marges
Clôture de caisse	ClotureCaisse, Ticket.mode_paiement/site_id	cloture.py	/cloture	Clôture
Santé caméras	Camera	—	/cameras, /events/heartbeat	Caméras
Journal d'audit	JournalAudit	audit.py	/audit	Journal d'audit
Objectifs & écarts	Objectif	objectifs.py	/objectifs	Objectifs
Alertes	Notification	notifications.py	/notifications	Alertes
Export comptable	—	—	/export/*.csv	Export
Idempotence ingestion	EvenementTraite	(ingestion)	(/events)	—

Points d'intégration notables :

- **Alertes** : `notifier_anomalie` est appelé dans `ingestion.finaliser_transaction`

pour toute anomalie `*haute/critique*` ; `notifier` dans la route clôture si écart.

Les canaux réels (SMTP, API WhatsApp) sont dans `services/notifications.py`

(gated par la config ; statut « simulé » si non configuré). L'envoi est **différé**

hors du chemin des requêtes : `notifier()` enregistre l'alerte en « en_attente », et un job du scheduler (`envoyer_notifications_en_attente`, toutes les `alert_dispatch_interval_s`) la transmet puis met à jour son statut (`envoye/echec`). `TODO(dev)` : compteur de tentatives + backoff pour re-tenter.

- **Audit** : `journaliser(db, user, action, ...)` instrumente les routes sensibles (annulation/rapprochement ticket, prix forfait, résolution anomalie, clôture, mouvement/suppression de stock).
- **Idempotence** : `EventIn.event_id` (uuid généré par l'edge) déduplique les re-livraisons de la file locale ; voir `traiter_evenement`.

16. i18n (FR / AR)

`frontend/src/i18n/{fr,ar}.js` (mêmes clés), `context/I18nContext.jsx`
`(useI18n() → { t, lang, setLang, dir })`, `components/LangSwitcher.jsx`. L'arabe bascule `<html dir="rtl">`. Toute nouvelle chaîne : `t("cle")` + les **deux** dictionnaires (un contrôle de parité existe : voir docs/).